STANDARD OPERATING PROCEDURE

NUScope-TOMO

FIB-SEM slice-and-view tomography: from raw slices to quantified, chemically resolved 3D microstructure

Created by Roberto dos Reis (@rmsreis) · NUANCE Center, Northwestern University

Document ID	NUSCOPE-TOMO-SOP-001
Revision	1.0 draft
Issue date	2026-11-01
Status	Draft for review. Not in force until the approval block below is signed.
Applies to	NUScope-TOMO 0.3.2 (PyPI `nuscope-tomo`, tag v0.3.2)
Author	Roberto dos Reis (@rmsreis)
Repository	github.com/NUANCE-IT/NUScope-TOMO
Review cycle	At every minor software release, and whenever a procedure step changes

Approval

Role	Name	Signature	Date
Prepared by			
Technical review			
Approved by			

Revision history

Rev.	Date	Change
1.0 draft	2026-11-01	First issue, written against software 0.3.2. Commands checked as listed in Appendix F. Section 17 covers the EDS fix made after 0.3.2.

Contents

1. Purpose and scope . . . 3
2. Roles . . . 3
3. Rules that apply to every run . . . 3
4. Definitions . . . 4
5. Installation and verification . . . 4
6. Workflow and choice of interface . . . 5
7. Procedure A: project setup and data intake . . . 6
8. Procedure B: data health check . . . 7
9. Procedure C: pre-processing . . . 8
10. Procedure D: segmentation . . . 9
11. Procedure E: quantification . . . 11
12. Procedure F: materials analyses . . . 12
13. Procedure G: uncertainty and representative volume . . . 12
14. Procedure H: reporting, comparison and archiving . . . 13
15. Procedure I: live acquisition . . . 14
16. Procedure J: acquisition planning . . . 14
17. Procedure K: correlative EDS . . . 15
18. Procedure L: self-driving runs . . . 16
19. Procedure M: use through NUANCE-MCP . . . 17
20. Troubleshooting . . . 18
21. Known limitations in 0.3.2 . . . 18
22. Worked example: sample 335 . . . 19
Appendix A. Recipe template . . . 20
Appendix B. Operation reference . . . 22
Appendix C. Project folder layout . . . 25
Appendix D. Run record . . . 26
Appendix E. Command reference . . . 27
Appendix F. How this SOP was checked . . . 27

1. Purpose and scope

This SOP tells an operator how to take a FIB-SEM slice-and-view stack through NUScope-TOMO and finish with results that can be defended and reproduced: phase fractions, particle statistics, transport and network metrics, each with its uncertainty and a provenance record.

In scope	Out of scope
Importing and checking slice stacks; pre-processing; segmentation; quantification; materials analyses; uncertainty; reporting and archiving; live monitoring of an acquisition; acquisition planning; correlative EDS from exported element maps; digital-twin and shadow-mode runs.	Microscope operation, specimen preparation, milling recipes and instrument safety (use the facility's instrument SOPs). Electron tomography from tilt series (planned, not built). Native Oxford `.h5oina` and Bruker `.bcf` reading (planned, not built). Closed-loop control of a real microscope (Section 18).

2. Roles

Role	Responsible for
Operator	Runs the procedure, keeps the run record (Appendix D), and stops at each check box.
Reviewer	A second person who confirms the segmentation by eye, checks the voxel size against the acquisition log, and signs the run record before results leave the group.
Maintainer	Controls the installed version, runs the test suite after each upgrade, and updates this SOP.
Microscope operator	Controls the instrument. Live mode gives this person advice and never commands the microscope.

3. Rules that apply to every run

- **Raw data is read-only.** Keep the original slices untouched. NUScope-TOMO works on a Zarr copy in the project folder.
- **Set the voxel size yourself.** Always give `voxel_size_nm: [z, y, x]`, taken from the acquisition log. Never rely on detection (Section 7).
- **One analysis, one project folder.** Re-running a recipe or `watch` in the same folder overwrites earlier outputs without asking.
- **No number leaves without its uncertainty.** Report the sampling interval and the boundary sensitivity (Section 13).
- **Inspect the segmentation before measuring it.** A clean-looking table can come from a wrong segmentation.
- **Simulated results stay labelled as simulated.** Digital-twin times and benchmark EDS scores are not experimental measurements.
- **Load only models you trust.** Random-forest models are Python pickle files, and loading a pickle can run code. Do not load one received from an unknown source.
- **Keep Studio private.** Studio has no login. Anyone who can reach its port can read data and start jobs (Section 6).

4. Definitions

Term	Meaning
Voxel size	Spacing in nm, always in the order [Z, Y, X]. Z is the slice thickness (milling axis).
Project	A folder holding volumes, tables, figures and provenance for one analysis.
Recipe	A YAML file listing operations in order. It is the reproducible definition of an analysis.
Operation (op)	A registered processing step, such as `preprocess.align`. Appendix B lists all of them.
Scribbles	A few brush strokes per class that train the classifier.
Bands	Intensity ranges used to bootstrap training labels automatically, with no human annotation.
Integral range	A length-scale measure from the phase covariance. It sets how much a phase fraction would vary between fields of view of the same size.
RVE	Representative volume: the volume needed to reach a target precision on a metric.
Boundary sensitivity	How much a metric changes if every phase boundary moves by one voxel.
Geodesic tortuosity	Shortest-path ratio through a phase, reported by `analyze.phases`.
Tortuosity factor	From a steady diffusion solve in `analyze.transport`. A different quantity from geodesic tortuosity; do not interchange them.
TPB	Triple-phase boundary: the line where three phases meet.
Digital twin	A simulated microscope built from an already measured stack.
Shadow mode	The agent runs on the twin while recording the AutoScript calls it would have sent.

5. Installation and verification

Requirements

- Python 3.11 or newer. Python 3.10 will not work, because Zarr 3 needs 3.11.
- A workstation with enough disk for the Zarr copy plus derived volumes. Most slice-wise steps stream from disk, so stacks larger than RAM are fine.
- Connected components, watershed and surface meshing run in memory. For a very large stack, bin it or work on a region of interest first.
- A GPU (Apple MPS or NVIDIA CUDA) is recommended for Noise2Void and the U-Net.
- Any modern browser with WebGL2 for Studio. The browser can be on another machine.

Install

```
pip install nuscope-tomo           # core
pip install "nuscope-tomo[ai]"     # adds PyTorch for Noise2Void and U-Net
```

Developers working in a clone:

```
uv pip install -e ".[dev]"
```

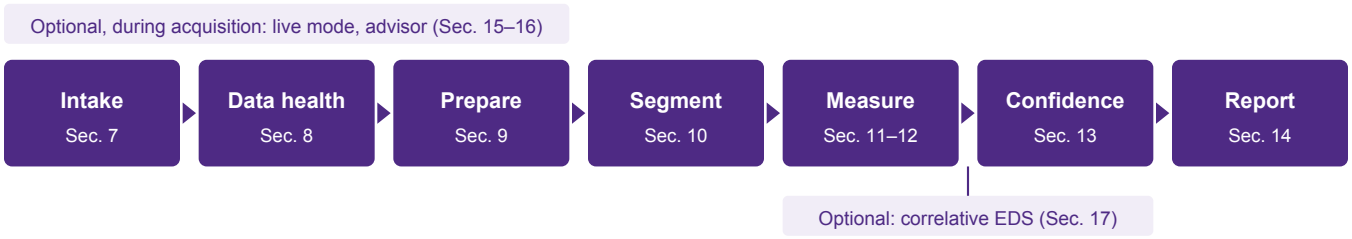
Verify

1. Run the two commands below. The first must print the six subcommands; the second must list the operations.
2. If you work in a clone, also run the test suite. Every test must pass: 29 on the 0.3.2 release, 30 once the EDS fix is included. They run on synthetic phantoms in under a minute.
3. Record the version in the run record.

```
nuscope-tomo --help
nuscope-tomo ops
python -m pytest -q                # clone only
```

> **Note.** With the NUScope platform package installed, the same commands are available as `nuscope tomo ...`. This SOP uses `nuscope-tomo`, which needs nothing else.

6. Workflow and choice of interface



The sequence is the same in every interface. Use a recipe for anything that will be reported, because a recipe is what can be re-run.

Interface	Use it for	Record it leaves
Recipe + CLI	Final analyses, batches, anything reported	`provenance.json` with the recipe, parameters, timings and library versions
Studio	Viewing, annotation, parameter exploration, review	`studio_recipe.yaml`, a log of actions. It cannot be run as is (Section 21)
Python API	Custom analysis and the EDS workflow	Your own script; keep it with the project
NUANCE-MCP	Asking an LLM to inspect a project or run a recipe	Recipe runs write the same outputs as the CLI (Section 19)

Starting Studio

```
nuscope-tomo studio projects/my_sample              # then open http://127.0.0.1:8765
nuscope-tomo studio projects/my_sample --port 8770  # if 8765 is taken
```

The left rail holds nine workspaces: Home, View, Prepare, Segment, Measure, Confidence, Chemistry, Acquire and Recipe. Press Cmd/Ctrl+K to search pages, data and operations. In the viewer, the keys 1–4 change the layout, F fits the view, and the arrow or Page keys step through slices.

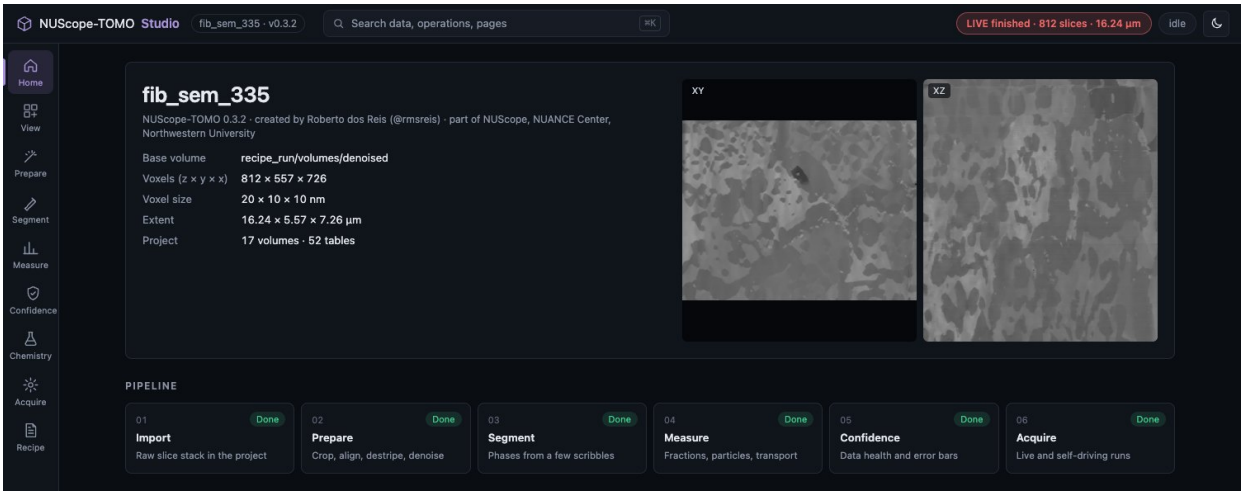


Figure 1. Studio Home for sample 335: project summary and pipeline status.

Using Studio from another machine

Keep Studio bound to 127.0.0.1 on the workstation and forward the port over SSH. The browser receives only slices and a downsampled 3D preview.

```
ssh -L 8765:127.0.0.1:8765 <user>@<workstation>     # run on the laptop
```

Warning. `--host 0.0.0.0` exposes Studio to the whole network, and Studio has no login. Anyone who reaches the port can read project data and start jobs. Use it only on a network you control.

7. Procedure A: project setup and data intake

1. Copy the slices to the workstation and make the original copy read-only.
2. Confirm from the acquisition log: slice thickness, pixel size, detector, stage tilt, and whether the stack was already registered or destriped by the acquisition software.
3. Start a run record (Appendix D).
4. Create a project folder and work from it. Paths for scribbles, models and exports are resolved against the folder you run from, so stay in one place.
5. Copy Appendix A into `recipe.yaml` and set the input path and `voxel_size_nm`.
6. For the first run, keep only the input block and the `qc.inspect` step, then run it. This imports the stack to `raw.zarr` and produces the data health tables for Section 8.
7. Check the import with `nuscope-tomo info`.
8. Add the remaining steps as you complete Sections 9 to 13, re-running as you go. To skip the import on later runs, change the input to `zarr: raw.zarr`, keeping `voxel_size_nm`.

```
cd projects/sample_12
nuscope-tomo run recipe.yaml                # first run: input and qc.inspect only
nuscope-tomo info raw.zarr                  # shape, voxel size, file count
```

Input key	Accepts	Notes
`tiff_folder`	A folder of TIFF, PNG, BMP or JPEG slices	Sorted by the trailing number in the file name, then streamed to `raw.zarr` in the project
`tiff_stack`	One multipage TIFF	Read directly

Input key	Accepts	Notes
`zarr`	An existing NUScope-TOMO Zarr volume	Use it to re-run without importing again

Warning. Voxel size detection is not reliable. With no `voxel_size_nm`, a Thermo Fisher pixel-size tag sets X, Y **and Z** to the pixel size, which is wrong for almost every slice-and-view stack. With no tag at all, every axis silently becomes 1 nm and no error appears. Every physical result then scales wrongly.

Check before moving on. Slice count matches the log. The first and last slices are the expected ones. `info` reports the voxel size you intended, in Z, Y, X order.

8. Procedure B: data health check

Run `qc.inspect` on the raw data before any processing. It is the first step in Appendix A. In Studio, use Confidence, then **Check data health**.

Output	What it tells you	Action
Flagged slices (`flags.csv`)	Robust outliers more than 5 MAD from a running median, in brightness, SNR, sharpness, curtaining or slice difference	View each flagged slice. Most are mild. Investigate clusters, and exclude or re-acquire damaged regions with `preprocess.crop`.
Blank or **duplicate**	Almost no contrast, or nearly identical to the previous slice	Treat as a real acquisition fault. Check against the log and do not interpolate over it silently.
SNR and **sharpness** trends	Falling trends suggest charging, defocus or a growing trench shadow	Note the affected range in the run record.
Brightness drift	Percent change between the first and last 20 slices	Handled by `normalize_slices`; note large values.
Curtaining index	Vertical against horizontal stripe strength, about 1 means balanced	Rising values mean destriping matters. Verify it in Section 9.
Calibration warning	Features are at least 1.8 times longer or shorter along Z than in-plane	Compare against the log and the known microstructure. See below.

Reading the calibration warning. The warning is a question, not a verdict. It compares correlation lengths, so a truly columnar or elongated structure triggers it as well as a wrong slice thickness. On sample 335 the check would have exposed the import metadata that recorded 50 nm. At the correct 20 nm it still reports a ratio of 2.1, which was accepted because the structure is genuinely elongated along Z. Record how you resolved it.



Figure 2. Confidence workspace: slice-by-slice data health for sample 335, with flagged slices in red.

> **Check before moving on.** Every flag cluster is explained. The calibration warning, if any, is resolved in the run record. The usable slice range is decided.

9. Procedure C: pre-processing

Run the steps in the order below. In Studio, select the base volume and use the seven-step wizard under Prepare; untick steps you do not need and edit parameters in place. For reported results, carry the final parameters into the recipe.

#	Operation	Purpose	When to change it	Check
1	`preprocess.auto_crop`	Removes Pt cap, trench walls and padding	If the crop is wrong, replace it with `preprocess.crop` and explicit ranges	Compare the shape before and after. **Mandatory**, see warning
2	`preprocess.align`	Sub-pixel drift correction	Set `roi` to a fiducial region if microstructure is inclined to the milling axis	Read `align.drift.csv`. View an XZ slice: edges must be continuous
3	`preprocess.destripe`	Removes curtaining	Raise `sigma` if stripes remain. Set `angle_deg` if they are not vertical	Compare before and after. Real vertical features must survive
4	`preprocess.flatten_shading`	Removes slow illumination gradients	Set `sigma_px` larger if it erases large real features	A phase should have the same grey level across the field
5	`preprocess.normalize_slices`	Removes slice-to-slice flicker	Increase `window` for strong flicker	An XZ view shows no banding
6	`preprocess.bin`	Averages pixels, reduces noise and size	Bin only XY unless Z is oversampled. Keep small features above about 4 pixels	New voxel size in `info`
7	`preprocess.denoise_nlm`	Non-local means denoising	Lower `h` if fine detail disappears	Thin features remain. Compare with the raw slice

Warning. Always check the crop. `auto_crop` looks for a block face and has no guard against a poor result. On a test stack with no trench it discarded more than half of the image height without any warning. The line `crop:` in `nuscope-tomo info` shows what was kept.

Honest alignment. Correlation cannot separate stage drift from microstructure that really is inclined to the milling axis, especially with thick slices. `align` therefore detects a stack that is already registered and leaves it alone. If `aligned: already registered` appears when you expected drift, or if alignment shears inclined features, use a fiducial `roi` or the `highpass_slices` drift model.

Other pre-processing operations

Operation	Use
`preprocess.denoise_n2v`	Noise2Void trained on this stack, with no clean target needed. Needs the [ai] extra. Prefer it when non-local means blurs fine structure.
`preprocess.denoise_tv`	Edge-preserving, gives piecewise-flat regions.
`preprocess.gaussian`, `preprocess.median`	Simple filters. Gaussian width is physical (nm) and respects anisotropy.
`preprocess.tilt_correct`	Corrects foreshortening of the cross-section at the SEM stage tilt (default 52°). Do not apply if the acquisition software already corrected it.
`preprocess.resample_isotropic`	Resamples to cubic voxels, for tools that need them. Analyses here do not need it, because they use physical spacing.
`preprocess.rescale_intensity`, `preprocess.unsharp`	Display and contrast adjustments. Do not use before intensity-based measurements.

Check before moving on. Crop verified. Curtains gone with real features intact. XZ and YZ views are continuous. No slice-to-slice banding. Final volume saved with `save: true`.

10. Procedure D: segmentation

Method	Choose it when	Notes
`segment.random_forest`	Default. Phases differ in grey level or texture	41 multi-scale features, trained from scribbles. Fast. Pair with `segment.smooth_labels`
`segment.unet`	The forest is confused by context	2.5D network, uses the forest's confident predictions as weighted pseudo-labels. Needs [ai] and a GPU in practice
`segment.multi_otsu`	Clearly separated grey levels	No training. Classes are ordered dark to bright
`segment.threshold`, `segment.hysteresis`	One binary phase	Otsu is used if no bounds are given. Hysteresis runs in memory

Train the random forest in Studio

1. Open **Segment**. Choose the pre-processed volume as the base volume.
2. Add one class per phase and name them. Names carry through to every table.
3. Paint a few strokes per class on XY slices. Spread them across the stack, including the top, middle and bottom, and cover each phase's difficult regions, such as boundaries and shadowed areas.
4. Click **Save**, then **Train & segment** with *Random forest (fast)*.

5 Overlay the result and view it in XY, XZ and YZ. Add strokes where it is wrong and train again.

6 Stop when further strokes no longer change the phases you care about.

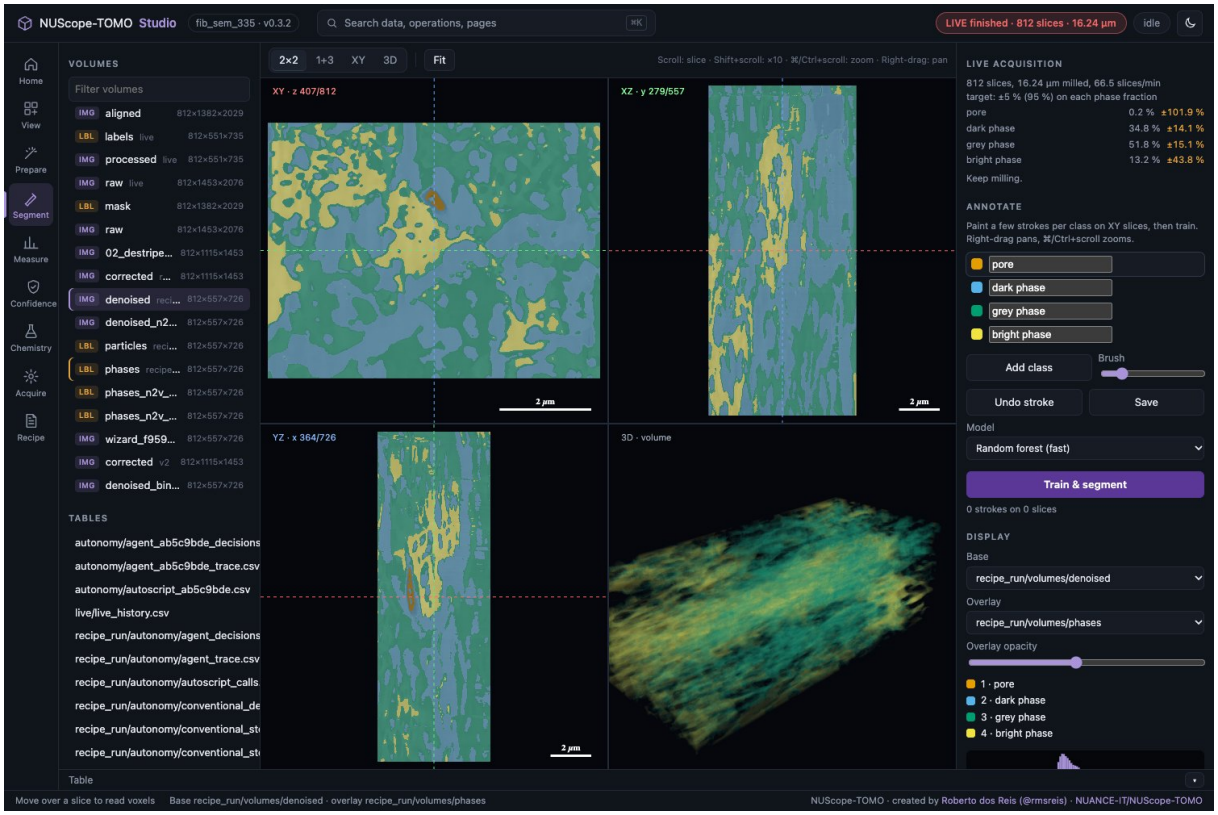


Figure 3. Studio viewer with phases overlaid on sample 335 in linked XY, XZ, YZ and 3D views.

Carry the training into the recipe

Studio saves strokes to `annotations/<volume id>.json` in the project, with any / in the id replaced by __. Point `scribbles` at that file. Each stroke has this form, with points in pixel coordinates of the volume being segmented:

```
[{"class": 1, "z": 120, "radius": 5, "points": [[y0, x0], [y1, x1], ...]}, ...]
```

Warning. Scribbles are stored in pixel coordinates, so they fit only a volume with the same crop and binning. If pre-processing changes the shape, annotate again.

Note. `bands` bootstrap training labels from intensity ranges with no human annotation. The sample 335 recipe uses them. They are useful for a first pass, but a reported segmentation needs scribbles or a documented visual review, because the result can only be as good as the bands.

Save the trained model

Set `model_path` on the segmentation step. A saved forest is what `nuscope-tomo watch` uses for live classification. The path is resolved against the folder you run from.

Review

> **Check before moving on.** A second person has viewed the overlay in all three planes at the top, middle and bottom of the stack. Thin films and small particles are not lost. Trace classes are real rather than noise. Class names are correct.

11. Procedure E: quantification

Phase metrics

`analyze.phases` writes volume fraction, surface area, specific surface, Euler number, connected clusters, percolation by axis, geodesic tortuosity, interfaces and a continuous size distribution.

Separating and measuring particles

1. Extract one class with `segment.extract_class`. Use `open_nm` and `min_size_nm3` to remove thin bridges and specks.
2. Split touching particles with `segment.separate`. Set `min_distance_nm` near the smallest real particle radius and raise `h_nm` if particles are over-split.
3. View the labels. Over-splitting and under-splitting both show up plainly in 3D and are the main source of error here.
4. Measure with `analyze.objects`. Pass `intensity` to add mean grey level per object.

Columns	Meaning
`volume_um3, surface_area_um2`	Volume from voxel count; marching-cubes surface area
`eq_diameter_nm, sphericity, solidity`	Equivalent-sphere diameter; 1 is a sphere; volume over convex hull
`feret_max_nm, axis_major/mid/minor_nm`	Maximum caliper; equivalent-ellipsoid axes
`elongation, flatness`	Axis ratios (mid/major, minor/mid)
`theta_deg, phi_deg`	Major-axis polar angle from the milling (Z) direction, and azimuth
`nn_distance_nm, centroid_*_um`	Nearest-neighbour distance; centroid position
`touches_border`	True if the object is cut by the volume boundary

> **Warning.** Exclude `touches_border` objects from size and shape statistics. They are truncated and bias the distribution towards small sizes. On sample 335, only 893 of 1,414 separated particles lie fully inside the volume. `nuscope-tomo compare` and Studio's size charts exclude them automatically; your own plots must do it too. `segment.remove_border_objects` drops them from the label volume.

> **Check before moving on.** Separated particles match the images. Object count is sensible. Reported statistics exclude border-touching objects.

12. Procedure F: materials analyses

Operation	Reports	Set	Watch for
`analyze.tpb`	Total and active TPB length and density	`phases`: the three class numbers	Length density in μm^{-2}. Active means on connected networks.
`analyze.transport`	D_{eff}/D_0, tortuosity factor and MacMullin number, per phase and axis	`phases`, `axes`, `max_voxels`	The field is downsampled to fit `max_voxels`. Confirm `converged` is True and read the `voxel_nm` and `grid` columns.
`analyze.network`	Nodes and links: contact or throat sizes, coordination numbers	Run on separated labels. `min_contact_um2` filters tiny contacts	Coordination depends strongly on the contact threshold; report it.

1. Add the analyses you need to the recipe, using `inputs: {labels: phases}` so they use the phase map, not the last volume.
2. Run a coarse transport solve, then repeat at a higher `max_voxels`. If the result moves, keep refining. The sample 335 recipe uses 25 million voxels (a grid near 30 nm).
3. Report the direction. Transport is often strongly anisotropic. Z is the milling axis.

> **Note.** The transport solver covers steady-state diffusion and conduction only. Permeability, fibre tracing and digital volume correlation are not available in this release.

13. Procedure G: uncertainty and representative volume

Every reported metric needs two error sources, because they answer different questions.

Source	Question	How it is obtained	Output
Sampling	Would another field of view of this size give the same value?	Integral range for fractions, block bootstrap for specific surface and tortuosity	`sampling_ci95_low/high`
Boundary sensitivity	How much does the value depend on exactly where the boundary sits?	All phase boundaries moved by one acquired voxel	`boundary_low/high`

1. Add `analyze.rve` and `analyze.uncertainty` to the recipe, or use Confidence, then **Uncertainty & volume needed**.
2. Read `rve.table.csv`. `n_rve_in_volume` below 1 means the volume is smaller than the RVE, and `rel_error_full_volume` gives the actual precision.
3. Compare with the project's target precision, set before the analysis. The software default is ±5 % at 95 %.
4. If the target is not met, either report the wider interval as it stands or acquire more volume. Do not claim the target.
5. Report each metric as value, 95 % sampling interval, boundary range.

Decision	Rule
Accept	Sampling interval meets the project target and the boundary range is no wider than the difference you need to resolve.
Accept with limits	Volume is below RVE. Report the true interval and state that the value applies to this volume, with stated precision.
Do not report	The segmentation failed review, the voxel size is unresolved, or the transport solve did not converge.

Note. These decision rules are this SOP's defaults. The numeric target belongs to each project, and the software does not enforce it.

Warning. Sampling error assumes the material is statistically uniform across the sampled scale. A real gradient across the volume, or a phase with sparse large features, makes the interval optimistic. Look for such trends in the data before relying on it. On sample 335, the 13 % bright phase is known only to ±54 %, which shows how quickly precision falls for minor phases.

14. Procedure H: reporting, comparison and archiving

Run the final recipe

```
cd projects/sample_12
nuscope-tomo run recipe.yaml
```

Compare samples

`compare` expects each project to contain `volumes/phases.zarr` and, for particle statistics, `tables/particles_measured.csv`. Use those exact step ids in the recipe, with `save: true` on phases.

```
nuscope-tomo compare A=projects/sample_a B=projects/sample_b --out cohort
```

The output folder holds `tests.csv`, `summary.csv` and `cohort.png`. Differences are tested against the sampling error, particle sizes use Mann–Whitney and Kolmogorov–Smirnov tests with Cliff's delta, and p-values are Holm-corrected. **A non-significant result does not show the samples are the same.** Read the minimum detectable difference in the output to see whether the volume could have detected the effect.

Figures

Studio's viewer gives quick views. For publication figures and turntable movies, use `nuscope_tomo.viz`; `examples/showcase.py` is a working template. Figures keep the true aspect ratio and scale bars.

What to report

- Sample, instrument, detector, voxel size, stack size and physical volume.
- Software version and recipe, with any change from Appendix A.
- Each metric with its sampling interval and boundary range, and for transport, its direction and grid.
- Citation: NUScope-TOMO by Roberto dos Reis, using the repository's CITATION.cff.

What to archive

Item	Why
Original slices and acquisition log	The only true source
`recipe.yaml`, scribbles JSON, trained model	Needed to reproduce the segmentation
`provenance.json`	Exact parameters, timings and library versions
`tables/` and final `volumes/`	The reported results
Signed run record (Appendix D)	Shows who checked what

15. Procedure I: live acquisition

`nuscope-tomo watch` follows the folder that Auto Slice & View writes to and analyses each slice as it arrives. It only reads files. **It never commands the microscope**, and its stop message is advice for the microscope operator.

1. Train a forest on a similar sample, or on the first slices of this one, and save it with `model_path`. Train on data processed like live data: 8-bit, binned by 2, denoised, as in Appendix A.
2. Start the watcher before or during milling, with the true voxel size in Z, Y, X order.
3. **Read the first line it prints**, which gives the block-face region fixed from the first 8 slices. Stop and restart if it is wrong.
4. Open Studio on the same project to watch the growing volume, or read the log. Statistics update every 25 slices once at least 16 slices are in.
5. When it reports *precision target reached, milling can stop*, the microscope operator decides. Phases under 1 % do not hold up this message.
6. The session ends after the idle time (60 s by default). After milling, run the full recipe on the complete stack. Live results are a monitor, not the final analysis.

```
nuscope-tomo watch /acq/run_12 -p projects/run_12 --voxel 20 5 5 --model rf.pkl --target 0.05 --idle 300
```

Option	Meaning	Default
`--voxel Z Y X`	Voxel size in nm. Required	none
`--model`	Saved forest. Without it, you get processed slices and no phase statistics	none
`--target`	Relative precision at 95 %	0.05
`--idle`	Seconds without a new slice before finishing. Set it longer than a full slice cycle plus any planned pause	60

Warning. The block-face region is set once, from the first 8 slices, and is not checked afterwards. On a test stack with no real trench, it selected a strip 10 pixels wide, and the phase fractions that followed were meaningless, with no error shown. If the first numbers look implausible, stop and fix the crop.

Warning. Starting `watch` again on the same project overwrites `<project>/live`. Use a new project after any interruption, or copy the live folder first.

Outputs are in `<project>/live`: `raw.zarr`, `processed.zarr`, `labels.zarr`, `live.json` and `live_history.csv`.

16. Procedure J: acquisition planning

`advise.acquisition` answers two questions from a segmented stack: is the sampling finer than the result needs, and how much volume does the target precision need?

1. Segment the first slices of a run, or a similar sample.
2. Add `advise.acquisition` with `inputs: {labels: phases}`, or use Confidence, then **Acquisition advice**.
3. Read the `advice` column in `advisor.recommendation.csv`.
4. Check `volume_estimate_reliable`. It stays False until the milled depth reaches 10 mean chord lengths, and early volume estimates can be far off.
5. Treat the result as a proposal. Coarser sampling is safe only for the metrics it tested: fractions, specific surface and chord length. If the study depends on small features or thin films, keep the finer setting.

Parameter	Default	Meaning
`tolerance`	0.05	Largest allowed change in tested metrics when sampling is coarsened
`target_rel_error`	0.05	Target precision for volume requirements
`px_per_feature, slices_per_feature`	4, 3	Pixels and slices a feature should span

17. Procedure K: correlative EDS

Warning. In the released 0.3.2, the `chem.eds_tomography` operation does not work. It always stops with `AttributeError: 'EDSStack' object has no attribute 'shifts_px'`, from a recipe or from Studio's Chemistry workspace. The bug is fixed in the source after commit 11f47bf and will ship in the next release. With 0.3.2 from PyPI, use the Python route below. With a build that includes the fix, use the recipe step.

Warning. This workflow has been checked only on simulated EDS maps, because no real EDS dataset has been processed yet. Validate it on a sample with known composition before reporting real results.

Prepare the maps

Export one image per EDS slice, one folder per element. Native `.h5oina` and `.bcf` files are not read in this release.

```
eds/Ni/map_000.tif  map_001.tif ...
eds/Mn/map_000.tif  map_001.tif ...
eds/O/map_000.tif   map_001.tif ...
```

Run the analysis: recipe step (versions after 0.3.2)

Add this step after the phases in Appendix A. Here `eds_voxel_nm` is the EDS voxel size in Z, Y, X, and `eds_every` is the number of SE slices between EDS maps.

```
  - op: chem.eds_tomography
    id: eds
    inputs: {volume: denoised, labels: phases}
    params: {eds_root: eds, eds_voxel_nm: [200, 30, 30], eds_every: 10, n_phases: 3}
```

It writes `eds.phases_table.csv`, `eds.registration.csv` (shift per EDS slice) and `eds.composition_by_class.csv`. Add `save: true` to keep the chemical phase map.

Run the analysis: Python (0.3.2 and later)

```
import numpy as np
from nuscope_tomo import Volume, VoxelSize
from nuscope_tomo.chem.eds import (read_eds_folders, register_to_se, denoise_counts, composition,
                                   chemical_phases, composition_by_label)

EVERY = 10                                              # one EDS map every 10 SE slices
se = Volume.from_zarr("volumes/denoised.zarr")          # target grid
phases = Volume.from_zarr("volumes/phases.zarr")        # same grid as se

eds = read_eds_folders("eds", VoxelSize(200, 30, 30))  # EDS Z, Y, X in nm
eds.z_index = np.arange(eds.shape[0]) * EVERY           # SE slice of each EDS map
reg = denoise_counts(register_to_se(eds, se), 30.0)    # map onto SE grid, then smooth (nm)
comp = composition(reg)
labels, table = chemical_phases(comp, 3, "kmeans")     # or "gmm", "nmf"
print(table)
by_phase = composition_by_label(comp, phases.numpy(), phases.meta.get("classes"))
print(by_phase)
```

1. Check that the EDS field of view matches the SE field, or that the mapping is right. A wrong field of view shifts every composition.
2. Compare a registered element map against the SE slice by eye.
3. Keep compositions relative unless k-factors have been supplied to `composition`. These are not standards-based quantification.
4. Interpret chemistry with EDS interaction volume in mind. Element maps are coarser than SE images, so thin features are blurred.

Note. Sparse deep kernel learning and coupled-NMF pan-sharpening exist as benchmarks on simulated chemistry (`examples/sparse_eds.py`, `examples/chemical_fusion.py`). SE-only context cannot see chemistry that is invisible in SE, so include BSE context. Treat the benchmark scores as method comparisons, not as performance on real samples.

18. Procedure L: self-driving runs

Warning. The AutoScript backend has never been run on a microscope. Its calls were checked against OpenFIBSEM source code only. Do not connect it to an instrument under this SOP. A first hardware session needs its own written plan, the instrument owner's approval and an operator at the microscope.

The agent mills, refocuses, corrects drift, adapts dwell and slice thickness, takes sparse EDS and stops at the requested precision. Under this SOP, it runs in simulation only.

Stage	Runs on	Touches beam?	Status under this SOP
Digital twin	A simulator built from a measured stack	No	Permitted
Shadow mode	The twin, recording the AutoScript calls	No	Permitted; review the recorded calls
Hardware dry run	`AutoScriptFIBSEM(dry_run=True)`: records calls without sending	No	Permitted
Armed hardware run	A real DualBeam	**Yes**	**Not permitted** without a separate approved plan

Run on the twin

1. In Studio, open **Acquire**. Choose the policy, clean volume, phase map, classifier, target error, depth limit and the phases that must reach the target.
2. Click **Start twin run**. Use **Stop after this slice** to end early; the current slice finishes first.
3. Compare *Self-driving agent* with *Fixed recipe* on the same settings.
4. Review the decision log and the recorded calls in `autonomy/`.

The same run from a script (it expects Noise2Void outputs and models from the earlier example scripts):

```
python examples/self_driving.py agent projects/fib_sem_335/recipe_run
```

Built-in safeguards

Safeguard	Default
Dry run	On. Calls are recorded, not sent
Arming	Refuses to act unless the operator supplies the exact phrase: *I am at the microscope and accept autonomous milling*
Maximum slices / depth / run time	2,000 slices / 50 µm / 24 h
Slice thickness range	5–200 nm
Dwell range	0.1–20 µs
Stop file	Creating the file ends the run after the current slice (`autonomy/STOP` in Studio)

Note. Twin results, such as the 2.37 h against 8.43 h shown in Studio for sample 335, are simulated. The twin's time model has not been calibrated against a real slice cycle, and its default EDS compositions are placeholders. Do not quote them as measured savings.

19. Procedure M: use through NUANCE-MCP

With NUANCE-MCP installed, an LLM client can inspect projects and start recipes. The tools do not touch the microscope.

Tool	Purpose
`tomo_project`	Volumes, voxel sizes, phases, tables and figures in a project
`tomo_table`	Reads a result table, with summary statistics
`tomo_volume`	Shape, voxel size, physical size and metadata of a volume
`tomo_phase_fractions`	Fractions, ±sampling standard error and surface density of a labelled volume
`tomo_ops`	Available operations and defaults
`tomo_run`	Starts a recipe in the background. Writes to the project folder
`tomo_job`	Follows a recipe run

Note. `tomo_run` can overwrite outputs in the project it is given (Section 3). All the rules of this SOP still apply, and a person must review results and sign the run record. Note that `tomo_phase_fractions` reports a standard error, not a 95 % interval.

20. Troubleshooting

Symptom	Likely cause	Action
All sizes wrong by a large factor, or extent reported as very small	Voxel size defaulted to the pixel size or to 1 nm	Set `voxel_size_nm` and re-run
`Unsupported input spec`	Recipe has no valid input key	Use `zarr`, `tiff_folder` or `tiff_stack`
`No result 'x'; available: [...]`	A step refers to an id that does not exist or comes later	Fix the `inputs` entry; give earlier steps an `id`
Unknown operation	Name is wrong, or copied from old docs	Check `nuscope-tomo ops`. Use `analyze.phases`, not `analyze.phase_metrics`
Analysis ran on the wrong volume	Without `inputs`, a step uses the last volume	State `inputs: {labels: phases}` explicitly
Scribbles, model or TIFF not found, or written somewhere unexpected	Those paths follow the folder you run from	Run from the project folder or use absolute paths
Crop removed real sample	`auto_crop` misjudged the block face	Replace it with `preprocess.crop`
Inclined features sheared after alignment	Correlation followed the microstructure	Use a fiducial `roi` or `highpass_slices`
`Level value of 5 is too high` warning	Image is small for wavelet level 5	Harmless on full-size data; lower `level` for small crops
Memory fills during separate, objects or meshing	These steps run in memory	Bin further or crop to an ROI
Transport shows `converged` False, or NaN	Phase does not percolate along that axis, or grid too coarse	Check `percolates_*`; raise `max_voxels`
`compare` cannot find phases	Label volume not saved under that name	Use id `phases` with `save: true`
Noise2Void or U-Net fails on import	PyTorch not installed	`pip install "nuscope-tomo[ai]"`
Studio port in use	Another server is running	Use `--port`
Studio job shows an error	The operation failed	Open the Jobs panel for the message
`'EDSStack' object has no attribute 'shifts_px'`	Bug in the released 0.3.2, fixed in the source	Use the Python route in Section 17, or a build with the fix
Live mode gives implausible fractions	Wrong block-face region or mismatched model	Read the first log line; restart in a new project

21. Known limitations in 0.3.2

Item	Effect	What to do
`chem.eds_tomography` crashes in the released 0.3.2	The operation cannot run from a recipe or from Studio	Python route, Section 17. Fixed in the source for the next release
Real EDS not yet exercised	EDS is checked on simulated maps only	Validate on a known sample
AutoScript backend untested on hardware	Self-driving is simulation only	Section 18
`studio_recipe.yaml` is a log	It has no input block and refers to Studio volume ids, so `nuscope-tomo run` cannot execute it. Studio jobs do not write `provenance.json`	Rewrite final steps as a recipe (Appendix A)
`auto_crop` and live ROI have no sanity check	A bad crop is silent	Mandatory checks in Sections 9 and 15

Item	Effect	What to do
Voxel size defaults are silent	Wrong physical units with no error	Always set it (Section 7)
No Studio login	Network exposure risk	Stay on 127.0.0.1 and use SSH forwarding
Overwrites without asking	Re-runs replace outputs	One project per analysis
Not available	Permeability, general 3D multi-volume registration, fibre tracing, DVC, native EDS file readers, tilt-series tomography	Use other tools; see PLAN.md

22. Worked example: sample 335

Sample 335 is a FIB-SEM SE stack of 812 slices at 20 × 5 × 5 nm, processed with `examples/fib_sem_335.yaml`. The results below come from the project's stored tables. The run recorded in `provenance.json` was made with version 0.2.0, and its 16 steps logged 400 s in total, or about 6.7 minutes on this workstation. The stack was already registered, so alignment made no changes.

Stage	Result
After crop and 2 × 2 bin	812 × 557 × 726 voxels at 20 × 10 × 10 nm, or 16.24 × 5.57 × 7.26 µm (657 μm^3 analysed)
Data health	34 of 812 slices flagged; median SNR 11.7; brightness drift 0.5 %; calibration ratio 2.1, resolved as a real elongation
Segmentation	Four classes by random forest, from intensity bands. Pore fraction 0.14 %
Particles (dark phase)	1,414 separated; 893 fully inside the volume
TPB	1.21 μm^{-2} total; 78 % on connected networks

Phase fractions with both error sources

Phase	Fraction	95 % sampling interval	Boundary ±1 voxel	Rel. error (95 %)	Cube edge for ±5 %
Dark	35.6 %	30.4–40.8 %	31.7–39.7 %	±15 %	17.8 µm
Grey	51.1 %	43.8–58.3 %	45.0–56.9 %	±14 %	17.4 µm
Bright	13.2 %	6.0–20.4 %	11.2–15.3 %	±54 %	42.8 µm

Reading it. The analysed volume is equivalent to a cube of about 8.7 µm, well short of the 17–18 µm needed for ±5 % on the major phases. Under Section 13, this is *accept with limits*: report the intervals as they stand and do not claim ±5 %.

Transport, tortuosity factor by axis (30 nm grid, all converged)

Phase	Z (milling)	Y	X	D_{eff}/D_0, Z
Dark	1.98	7.92	5.90	0.180
Grey	1.39	3.82	3.92	0.367
Bright	11.3	35.7	60.8	0.012

The same grey phase has a geodesic tortuosity of 1.03 along Z, against a tortuosity factor of 1.39. This shows why the two must not be interchanged.

Acquisition advice. With all 812 slices, the advisor reported that 10 nm pixels and 60 nm slices would give the same fractions, surfaces and feature sizes within 4.6 %, with three times fewer slices. After only 100 slices it could not yet say so and marked its volume estimate as preliminary. This is why early advice must be read with caution.

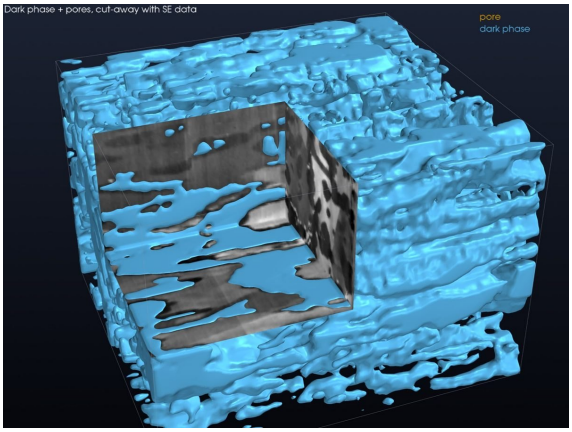


Figure 4. Sample 335: dark phase with a grey-value cut-away.

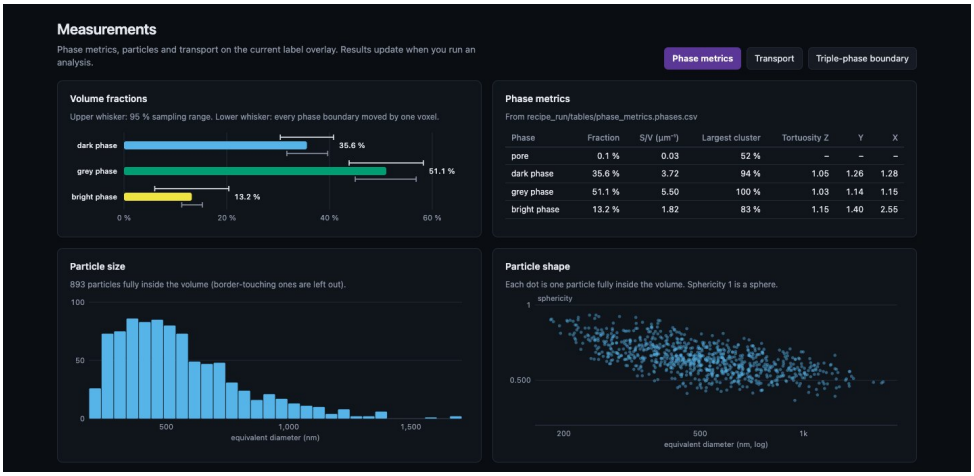


Figure 5. Measure workspace for sample 335.

Appendix A. Recipe template

This recipe was run from start to finish as printed for this SOP, apart from the input path. Change the marked lines. Keep the ids `phases` and `particles_measured` so that `compare` works. Remove analyses you do not need.

```
name: sample_12                              # CHANGE
project: .                                   # run from the project folder
input:
  tiff_folder: /data/sample_12/slices        # CHANGE
  voxel_size_nm: [20, 5, 5]                  # CHANGE: [slice thickness, pixel y, pixel x]
steps:
  # ---- data health on the raw stack -------------------------------
  - op: qc.inspect
    id: qc_raw
  # ---- pre-processing ---------------------------------------------
  - op: preprocess.auto_crop                 # check the crop afterwards
  - op: preprocess.align
    id: align
  - op: preprocess.destripe
    params: {level: 5, sigma: 3.0}
  - op: preprocess.flatten_shading
  - op: preprocess.normalize_slices
  - op: preprocess.bin
    params: {factors: [1, 2, 2]}
  - op: preprocess.denoise_nlm
    id: denoised
    params: {h: 0.8}
    save: true
  # ---- segmentation -----------------------------------------------
  - op: segment.random_forest
    id: rf
    params:
      classes: 3                             # CHANGE
      class_names: {1: pore, 2: matrix, 3: particle}   # CHANGE
      scribbles: annotations/volumes__denoised.json    # CHANGE
      model_path: rf.pkl
  - op: segment.smooth_labels
    id: phases
    params: {size: 3}
```

```
  save: true
# ---- quantification and confidence ------------------------------
- op: analyze.phases
  id: phase_metrics
  inputs: {labels: phases}
- op: analyze.rve
  id: rve
  inputs: {labels: phases}
- op: analyze.uncertainty
  id: uncertainty
  inputs: {labels: phases}
- op: advise.acquisition
  id: advisor
  inputs: {labels: phases}
# ---- particles --------------------------------------------------
- op: segment.extract_class
  id: part
  inputs: {volume: phases}
  params: {label: 3, open_nm: 20}          # CHANGE: class to separate
- op: segment.separate
  id: particles
  params: {min_distance_nm: 100, h_nm: 15} # CHANGE: from particle size
  save: true
- op: analyze.objects
  id: particles_measured
  inputs: {labels: particles, intensity: denoised}
# ---- materials analyses -----------------------------------------
- op: analyze.tpb
  id: tpb
  inputs: {labels: phases}
  params: {phases: [1, 2, 3]}              # CHANGE
- op: analyze.transport
  id: transport
  inputs: {labels: phases}
```

```
  params: {phases: [2, 3], max_voxels: 8000000}   # CHANGE
- op: analyze.network
  id: contacts
  inputs: {labels: particles}
# ---- export -----------------------------------------------------
- op: io.export_tiff
  inputs: {volume: phases}
  params: {path: phases.tif}
```

Recipe rules. A step takes the previous volume unless `inputs` says otherwise. `save: true` writes `volumes/<id>.zarr`; anything not saved disappears when the run ends. Tables are always written. Steps without an id are numbered, for example `03_destripe`.

Appendix B. Operation reference

Generated from the 0.3.2 operation registry. The right column shows default parameter values.

Input and output

Operation	Summary	Defaults
`io.export_tiff`	Write a volume as a (Big)TIFF stack with ImageJ-compatible calibration.	`path='export.tif', bigtiff=True`
`io.save`	Persist a volume as a chunked Zarr v3 store.	`path='volume.zarr'`

Quality control

Operation	Summary	Defaults
`qc.inspect`	Per-slice quality metrics, flagged slices and a calibration check.	`step_xy=2, z_score=5.0`

Pre-processing

Operation	Summary	Defaults
`preprocess.align`	Register slices (sub-pixel phase correlation, robust drift model).	`method='multibaseline', patch=1024, upsample=10, roi=None, highpass_slices=0.0, skip_if_registered=True, subpixel=True, crop_valid=True, outlier_mad=5.0`
`preprocess.auto_crop`	Crop to the block face: remove Pt cap, trench walls and padding.	`margin=20, edge_frac=0.5`
`preprocess.bin`	Block-average binning (noise reduction + size reduction).	`factors=(1, 2, 2)`
`preprocess.crop`	Crop to explicit index ranges [start, stop) per axis.	`z=None, y=None, x=None`
`preprocess.denoise_n2v`	Noise2Void self-supervised deep-learning denoising (trained on this stack).	`steps=1500, patch=64, batch=64, model_path=None`
`preprocess.denoise_nlm`	Non-local means per slice; ``h`` is relative to the estimated noise σ.	`h=0.8, patch_size=5, patch_distance=6`
`preprocess.denoise_tv`	Total-variation (Chambolle) per slice: edge preserving, piecewise flat.	`weight=0.05`
`preprocess.destripe`	Remove FIB curtaining stripes (wavelet–FFT filter).	`level=5, sigma=3.0, wavelet='db6', angle_deg=0.0`
`preprocess.flatten_shading`	Divide out the slow illumination gradient (shading correction).	`sigma_px=None`
`preprocess.gaussian`	3D Gaussian with a physical width (anisotropy-aware).	`sigma_nm=10.0`
`preprocess.median`	Median filter, per slice (``2d``) or in 3D.	`size=3, mode='2d'`
`preprocess.normalize_slices`	Remove slice-to-slice brightness/contrast flicker by matching each	`window=15, match='percentile'`
`preprocess.resample_isotropic`	Resample to cubic voxels (default: the coarsest axis spacing).	`voxel_nm=None, order=1`
`preprocess.rescale_intensity`	Stretch the robust intensity range to the full output dtype.	`low_pct=0.5, high_pct=99.5, dtype='uint8'`
`preprocess.tilt_correct`	Correct foreshortening of the cross-section imaged at the SEM stage	`stage_tilt_deg=52.0`
`preprocess.unsharp`	Unsharp-mask edge enhancement per slice.	`sigma_px=2.0, amount=0.6`

Segmentation

Operation	Summary	Defaults
segment.connected_components	Label connected objects of a binary mask (uint32 object ids).	min_size_nm3=0.0, connectivity=1
segment.extract_class	Binary mask of one class with optional opening / small-object cleanup.	label=1, min_size_nm3=0.0, fill_holes_nm3=0.0, open_nm=0.0
segment.hysteresis	Hysteresis threshold (seeds above ``high`` grown through ``low``) — in memory.	low=0.0, high=0.0
segment.multi_otsu	Multi-class Otsu: labels 1..classes ordered dark → bright.	classes=3, names=None
segment.random_forest	Train a random-forest classifier on sparse annotations and segment all slices.	scribbles=None, classes=3, class_names=None, bands=None, n_slices=6, sigmas=(1.0, 2.0, 4.0, 8.0, 16.0), n_estimators=100, model_path=None
segment.remove_border_objects	Drop objects touching the volume boundary (biased/truncated for statistics).	axes=None
segment.separate	Split touching particles: anisotropic distance transform + h-maxima seeds + watershed.	min_distance_nm=100.0, h_nm=15.0, min_size_nm3=0.0, compactness=0.0, seed_voxel_nm=None
segment.smooth_labels	Majority (mode) filter on a class map: removes salt-and-pepper voxels.	size=3
segment.threshold	Binary mask low <= I <= high (Otsu if no bounds given).	low=None, high=None
segment.unet	Deep-learning (U-Net) segmentation trained from sparse scribbles.	scribbles=None, classes=3, class_names=None, bands=None, n_slices=10, steps=2000, pseudo_labels=True, context=1, model_path=None

Analysis

Operation	Summary	Defaults
analyze.network	Pore / particle network model from separated regions (nodes + throats/contacts).	min_contact_um2=0.0
analyze.objects	Measure every labelled object (volume, area, shape, orientation, ...).	elements=None, max_objects=20000
analyze.phases	Volume fraction, specific surface, interfaces, percolation, tortuosity, size distribution per class.	tortuosity=True, max_voxels_tortuosity=884736, max_voxels=400000000
analyze.rve	Representative volume and whole-volume sampling error of each phase fraction.	phases=None, rel_error=0.05, max_voxels=8000000
analyze.tpb	Triple-phase boundary length density (total and active).	phases=(1, 2, 3), active='boundary', max_voxels=64000000
analyze.transport	Effective diffusivity / conductivity and tortuosity factor per phase and axis.	phases=None, axes=(0, 1, 2), max_voxels=8000000

Operation	Summary	Defaults
`analyze.uncertainty`	Value ± sampling and segmentation uncertainty for ε, S/V and tortuosity factor.	`phases=None, axis=0, n_blocks=6, max_voxels_transport=8000000`

Planning

Operation	Summary	Defaults
`advise.acquisition`	Recommend pixel size, slice thickness and volume from (early) segmented slices.	`phases=None, tolerance=0.05, target_rel_error=0.05, px_per_feature=4.0, slices_per_feature=3.0, xy_factors=(1, 2, 3, 4), z_factors=(1, 2, 3, 4), max_voxels=60000000`

Chemistry

Operation	Summary	Defaults
`chem.eds_tomography`	Import EDS element maps, register to SE, cluster chemical phases, fuse with labels.	`eds_root='', eds_voxel_nm=None, eds_every=1, n_phases=3, sigma_nm=30.0, method='kmeans'`

Appendix C. Project folder layout

```
<project>/
  raw.zarr                    Zarr copy of the slices (tiff_folder input)
  provenance.json             recipe, parameters, timings, library versions
  volumes/<id>.zarr           every step with save: true
  tables/<id>.csv             table outputs
  tables/<id>.<name>.csv      outputs of steps that produce several tables
  annotations/<volume>.json   Studio scribbles
  studio/                     Studio results
  studio_recipe.yaml          log of Studio actions
  autonomy/                   twin runs: trace, decisions, AutoScript calls, STOP file
  live/                       live mode stores, live.json, live_history.csv
```

Table (with Appendix A ids)	Contents
`qc_raw.summary / .slices / .flags.csv`	Data health
`align.drift.csv`	Measured drift per slice
`rf.feature_importance.csv`	Classifier feature ranking
`phase_metrics.phases / .interfaces / .size_distribution.csv`	Phase metrics
`rve.table / .curves.csv`	Integral range, RVE and precision
`uncertainty.csv`	Sampling intervals and boundary ranges
`advisor.recommendation / .feature_scale / .coarsening / .volume.csv`	Acquisition advice
`particles_measured.csv`	One row per object
`tpb.table.csv, transport.csv, contacts.nodes / .links.csv`	Materials analyses

Appendix D. Run record

Complete one record per project. The reviewer signs before results leave the group.

Sample / project ID	
Operator / date	
Instrument / detector	
Raw data location	
Software version	
Slices (log / imported)	
Voxel size Z, Y, X (nm), source	
Target precision	

Intake	☐ Raw data read-only, log reviewed ☐ Slice count matches ☐ Voxel size confirmed by `info`
Data health	☐ Flagged slices explained ☐ Calibration warning resolved or absent ☐ Usable slice range decided
Prepare	☐ Crop verified ☐ XZ and YZ continuous ☐ Curtains removed, real features intact ☐ Noise reduced, fine detail kept
Segment	☐ Scribbles spread through the stack ☐ Reviewed in three planes by a second person ☐ Model and scribbles saved
Measure	☐ Particle separation reviewed ☐ Border-touching objects excluded ☐ Transport converged and direction noted
Confidence	☐ Sampling and boundary errors recorded ☐ Decision: accept / accept with limits / do not report
Report	☐ Final results come from a recipe, not Studio alone ☐ Provenance, recipe, scribbles and model archived ☐ Simulated results labelled

Deviations from this SOP	
Operator signature / date	
Reviewer signature / date	

Appendix E. Command reference

```
nuscope-tomo info    <volume.zarr | image folder>
nuscope-tomo ops
nuscope-tomo run     recipe.yaml [-p project_dir]
nuscope-tomo studio  [project_dir] [--port 8765] [--host 127.0.0.1]
nuscope-tomo watch   <folder> -p project --voxel Z Y X [--model rf.pkl] [--target 0.05] [--idle 60]
nuscope-tomo compare NAME=project_dir [NAME=project_dir ...] [--out cohort]
```

`-p` on `run` overrides the recipe's `project`. If neither is given, the output goes to `projects/<recipe name>`.

Appendix F. How this SOP was checked

Written against commit 11f47bf (version 0.3.2) on 2026-11-01, on macOS with Python 3.12.

Item	Check	Result
Test suite	`pytest -q`	29 passed on 0.3.2; 30 with the EDS fix and its test
Appendix A recipe	Run end to end on a synthetic 120-slice TIFF phantom	Passed
`info, ops, run, compare`	Run on phantoms	Passed
`watch`	Run while slices were copied in, with a saved forest	Ran; bad ROI on the phantom (Section 15)
EDS Python route	Run on simulated maps	Passed
`chem.eds_tomography`	Run from a recipe on simulated maps	**Failed** on 0.3.2; passed with the fix (Section 17)
Studio	Started on sample 335; pages rendered for the figures	Passed
Intake-only recipe, then `info raw.zarr`	Run on a phantom	Passed
Agent, AutoScript dry run, U-Net, Noise2Void, MCP tools	Covered by the test suite only. Not separately exercised	Passed in pytest
Real EDS data, microscope hardware	Not available	**Not tested**
Studio wizard and training clicks; `nuscope tomo` plugin route	Procedure text comes from the source, not from a click-through	Not exercised

References

- NUScope-TOMO repository: README.md, PLAN.md, docs/IDEAS.md, CITATION.cff.
- Kanit et al. (2003), determination of the representative volume size (integral-range approach used by `analyze.rve`).
- Borodinov, Kalinin, Ovchinnikova et al. (2019), npj Computational Materials (pan-sharpening approach).
- Kalinin, Ziatdinov et al., deep kernel learning for context-driven measurement.
- OpenFIBSEM (DeMarco lab), source of the AutoScript call patterns.